

Rapport d'Audit de Sécurité

Projet : Application Mobile Nickel | Rôle : Consultant Sécurité

1. Résumé Exécutif

Ce document présente la synthèse de l'audit de sécurité technique mené sur l'application Android cible. L'objectif de cette mission était d'évaluer la robustesse de l'application face à l'ingénierie inverse et d'identifier d'éventuelles fuites de données. L'analyse a révélé plusieurs vulnérabilités critiques, notamment la compromission de la base de données cloud et la fuite de secrets cryptographiques codés en dur.

2. Méthodologie Employée

L'analyse a suivi une approche standard de pentest mobile, combinant analyse statique et dynamique :

- Rétro-ingénierie** : Décompilation de l'archive (APK) à l'aide d' `apktool` pour extraire le code Smali et les fichiers de ressources.
- Analyse Statique (SAST)** : Inspection des fichiers `strings.xml` et du Manifest via des requêtes `grep` ciblées pour identifier les clés d'API et les endpoints.
- Recompilation et Signature** : Assemblage d'un nouvel APK et signature via `keytool` et `apksigner` pour valider l'intégrité de la chaîne de compilation.
- Analyse Dynamique (DAST)** : Tests de requêtes HTTP directes sur les infrastructures tierces identifiées (Firebase).

3. Résultats de l'Audit (Findings)

3.1. Base de données Firebase exposée publiquement (Criticité : CRITIQUE)

Description : La base de données Firebase utilisée par l'application (Realtime Database) est accessible en lecture de manière non authentifiée. Les règles de sécurité par défaut n'ont pas été restreintes.

Preuve technique : L'exécution de la commande `curl -s "https://application-client-nickel.firebaseio.com/.json"` retourne un accès valide aux nœuds de données.

Impact : Risque maximal de fuite de Données à Caractère Personnel (PII). Un attaquant distant peut exfiltrer l'intégralité de la base de données en une seule requête, entraînant une violation majeure du RGPD.

Recommandation : Implémenter immédiatement des Firebase Security Rules strictes (ex: `".read": "auth != null"`) pour exiger un jeton d'authentification valide avant tout accès.

3.2. Fuite de secrets codés en dur (Criticité : MOYENNE)

Description : L'analyse du fichier de ressources `res/values/strings.xml` a mis en évidence la présence de multiples secrets et identifiants intégrés directement dans le code client.

Preuve technique : Extraction de variables critiques telles que `google_api_key` et `ACCOUNT_ENDPOINT` de production.

Impact : Ces clés permettent à un utilisateur malveillant d'usurper l'identité de l'application pour consommer des quotas d'API tiers payants ou d'attaquer directement les serveurs backend de production.

Recommandation : Ne jamais stocker de secrets dans le code client. Utiliser le système Android Keystore, obfuscater le code (ProGuard/R8), et déléguer l'authentification des API sensibles à un serveur proxy intermédiaire sous le contrôle de l'entreprise.

4. Conclusion

La surface d'attaque de l'application est actuellement très exposée. Le manque d'obfuscation et l'absence de règles de sécurité sur les infrastructures cloud rendent l'exploitation des vulnérabilités triviale pour tout attaquant disposant d'outils d'analyse standards. La priorité absolue doit être donnée à la sécurisation de l'instance Firebase.